

# Ciclo de Vida de una Activity

Guía de estados, callbacks y mejores prácticas en Android Studio

## ¿Qué es el Ciclo de Vida?

En Android, una **Activity** representa una pantalla con la que el usuario puede interactuar. A medida que el usuario navega por la app, sale de ella o regresa, la instancia de la actividad transiciona a través de varios estados. El sistema operativo notifica estos cambios mediante métodos *callback* que permiten ejecutar código específico según el estado de la pantalla.

## Métodos del Ciclo de Vida

| Método Callback          | Estado Resultante | Descripción y Responsabilidad                                                                                                                                        |
|--------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>onCreate()</code>  | Creado            | <b>Obligatorio.</b> Se ejecuta al crear la actividad. Aquí se inicializan layouts ( <code>setContentView</code> ), variables y vinculación de vistas (View Binding). |
| <code>onStart()</code>   | Iniciado          | La pantalla se vuelve visible para el usuario, pero aún no es interactiva. Preparación para pasar a primer plano.                                                    |
| <code>onResume()</code>  | Reanudado         | La actividad está en primer plano y lista para la interacción del usuario. Es el estado activo en el que permanece mientras se usa.                                  |
| <code>onPause()</code>   | Pausado           | Otra actividad toma parcialmente el foco (ej. una llamada o cuadro de diálogo). Se deben pausar animaciones o reproductores de medios.                               |
| <code>onStop()</code>    | Detenido          | La actividad ya no es visible. Se deben liberar recursos pesados, guardar datos persistentes o pausar sincronizaciones de red.                                       |
| <code>onRestart()</code> | Iniciado          | Se ejecuta cuando la actividad pasa de <code>onStop()</code> de nuevo a ser visible. Posteriormente invoca a <code>onStart()</code> .                                |
| <code>onDestroy()</code> | Destruído         | Se llama antes de que la actividad sea eliminada del todo (al presionar Atrás o cuando el sistema destruye el proceso por memoria).                                  |

### 💡 Regla de Oro en la Gestión de Recursos

Siempre complementa los callbacks: si inicias un recurso en `onStart()`, libéralo en `onStop()`. Si configuras algo en `onResume()`, paúsalo en `onPause()`.

## Ejemplo de Implementación en Kotlin

A continuación se muestra una clase MainActivity estándar registrando cada evento del ciclo de vida en Logcat:

MAINACTIVITY.KT

```
import android.os.Bundle
import android.util.Log
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private val TAG = "CicloDeVida"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d(TAG, "onCreate: La actividad se ha creado")
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart: La actividad es visible")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume: La actividad está interactiva (primer plano)")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause: La actividad pierde el foco")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop: La actividad ya no es visible")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, "onRestart: Reiniciando actividad a estado visible")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: La actividad se ha destruido")
    }
}
```